

CHAOS ENGINEERING AGENT

Complete Project Reference Document

What it is · How it works · Features · Fault catalogue · Usage · Scenario definition ·
Remediation · Build plan

Version 2.0 · June 2026

Internal Use Only · Confidential

1. What Is This Project?

The Chaos Engineering Agent is an AI-powered autonomous system that deliberately breaks distributed systems in controlled ways to discover resilience weaknesses before they become real production outages. It combines a Large Language Model reasoning layer with industry-standard fault injection tooling to make chaos engineering accessible, safe, and continuous for every engineering team.

Traditional testing cannot predict the complex failure modes that emerge in production. Real outages teach teams about weaknesses — but at the cost of customer impact and revenue loss. The agent flips this: it proactively finds weaknesses through controlled experiments before customers ever feel the pain.

1.1 The Core Problem

Problem	Impact
Reactive culture	Teams discover failure modes during real incidents. Customers feel the pain first.
High barrier to entry	Chaos engineering requires deep domain expertise. Only large platform teams with dedicated SREs can do it today.
Manual overhead	Planning experiments, writing chaos manifests, collecting metrics, and writing post-mortems takes days of skilled effort.
No learning loop	Findings from incidents rarely feed back into a systematic programme of resilience improvement.

1.2 The Solution

The agent

An engineer describes a failure scenario in plain English — or the agent generates one automatically. It simulates the experiment on a digital twin first, then executes the real experiment with adaptive intensity and full automatic rollback. It diagnoses root causes, prescribes specific fixes, and creates pre-filled tickets in GitHub or Jira. Every cycle makes the system measurably more resilient.

1.3 The Digital Twin Innovation

Before any fault fires against real infrastructure, the agent auto-maps the entire service graph from Kubernetes manifests, Istio topology, Prometheus metrics, and distributed traces. It runs 1,000+ Monte Carlo simulations to predict blast propagation paths and failure probabilities, and generates a written pre-mortem shared with the team. After the experiment, it compares reality vs prediction — the gap trains the twin to be more accurate next time.

Over hundreds of experiments, each service builds a Chaos DNA profile — a unique resilience fingerprint of its failure patterns, weak links, and recovery behaviours. New deployments are matched against historical profiles to get instant experiment recommendations.

1.4 Key Outcomes

Outcome	What it means in practice
Proactive resilience	Discover failure modes before customers do, in a controlled environment with automatic rollback
Zero expert dependency	Any engineer can run a chaos experiment via natural language — no YAML, no domain expertise required
Continuous improvement	Every experiment produces actionable tickets. Every ticket, when resolved, raises the resilience score
Production confidence	Teams deploy knowing the system has been tested against the most likely failure scenarios
Audit trail	Full experiment history, metric snapshots, and runbooks for compliance evidence (SOC 2, ISO 27001)

2. How It Works

The agent is a modular, event-driven system composed of four layers. Each layer communicates via well-defined interfaces, enabling components to be swapped or extended independently.

2.1 The Experiment Lifecycle

Step	Stage	What happens
1	Describe or select	Engineer describes a scenario, approves an agent suggestion, or CI/CD triggers automatically.
2	Twin simulation	Agent builds a digital twin and runs Monte Carlo simulations to predict blast radius and failure paths.
3	Pre-mortem	LLM generates a written prediction: what we expect, what metrics to watch, what the escape hatch is.
4	Plan generated	Structured experiment plan produced: fault type, scope, duration, intensity ramp, rollback procedure.
5	Human gate	In production mode, a Slack approval request is sent. In staging or CI mode, auto-approved.
6	Fault injected	Chaos manifest applied to Kubernetes. Adaptive intensity starts at 10% and ramps every 30 seconds.
7	Guard monitors	Steady-state guard evaluates metrics every 15 seconds against pre-captured baseline.
8	Rollback	On abort or completion, pre-validated rollback fires. Agent waits for metrics to return to baseline.
9	Diagnosis	LLM performs root cause analysis. Compares twin prediction vs actual. Identifies weakest link.
10	Remediation	Specific fixes prescribed. Pre-filled tickets created. Resilience score updated.

2.2 The Four Architecture Layers

Layer 1 — Orchestration

Python 3.12 and FastAPI. Accepts requests via REST API, CLI, or chat interface. Manages the experiment state machine. Uses Celery and Redis for async job execution and PostgreSQL for full audit trail.

Layer 2 — LLM Reasoning

Powered by Claude API and LangChain. Translates natural language into structured experiment plans, generates scenario suggestions, creates digital twin pre-mortems, performs root cause

analysis, and writes incident runbooks. Maintains full conversation context for iterative experiment design.

Layer 3 — Fault Execution

Translates plans into actual Kubernetes fault injections via Chaos Mesh CRDs and LitmusChaos. Toxiproxy handles dependency-level faults. AWS FIS and GCP FIS cover cloud-level failures. Blast radius controls enforced at the API level — cannot be overridden.

Layer 4 — Observability

Prometheus scrapes metrics every 15 seconds. Custom SLO tracker computes error budget burn rate in real time. Grafana dashboards auto-provisioned per experiment. PagerDuty and Slack webhooks fire on abort and completion events.

2.3 Safety Controls

Control	How it works
Steady-state guard	Auto-abort if error rate >2x or latency >3x pre-captured baseline. Evaluates every 15 seconds.
Blast radius control	Max 30% of pod replicas per deployment. Single namespace default. Human gate for production.
Idempotent rollback	Pre-validated rollback stored before every fault. TTL auto-cleans manifest if agent crashes.
Human approval gate	Production experiments require Slack one-click approve/reject. Cannot be bypassed via API.
Audit trail	Every action logged with timestamp and metric snapshot. Immutable after experiment closes.

3. Feature Overview

The agent has 18 core features across 5 capability areas, each addressing a specific gap in how engineering teams currently approach resilience testing.

3.1 AI Intelligence (4 features)

Feature	What it does
Digital twin simulation	Builds a live dependency graph and runs Monte Carlo simulations before any real fault fires. Prediction vs reality diff improves the twin over time.
Chaos DNA profiling	Each service accumulates a unique resilience fingerprint. New deployments get similarity-matched experiment recommendations instantly.
Adaptive fault intensity	Starts at 10%, ramps up or backs off every 30 seconds based on live metric feedback. Configurable speed: aggressive (CI) or conservative (prod).
Natural language planning	Engineers describe scenarios in plain English. Claude API translates to a structured experiment plan with safety validation. No YAML authoring.

3.2 Safety and Control (3 features)

Feature	What it does
Steady-state guard	Continuous metric evaluation every 15 seconds. Auto-abort and rollback if thresholds are breached.
Blast radius control	Hard limits enforced at API level: max 30% replicas, single namespace, human gate for production.
Idempotent rollback	Pre-validated rollback stored before every fault. Safe to call multiple times. TTL safety net if agent crashes.

3.3 Observability (3 features)

Feature	What it does
Real-time metric stream	WebSocket feed streams experiment state and metric snapshots live. Grafana dashboard auto-provisioned per experiment.
Resilience scorecard	Weekly 0-100 score per service tracked over 90 days. Score drops trigger automated experiment suggestions.
Auto-generated runbooks	Post-experiment Markdown runbooks pushed to Confluence or GitHub Wiki. Under 15 minutes of human editing required on average.

3.4 Integrations (3 features)

Feature	What it does
CI/CD chaos gate	GitHub Actions and GitLab CI run chaos smoke tests on every merge to main. Pipeline fails if resilience score drops more than 10 points.
Slack and PagerDuty	Real-time alerts for all experiment events. /chaos abort slash command. Auto-page on-call on unintended SLO breach.
Multi-cloud support	AWS FIS, GCP FIS, Chaos Mesh, and LitmusChaos under one unified experiment API.

3.5 Next-Level Roadmap Gaps

Gap	Why it matters
Incident-driven scheduling	Agent watches PagerDuty and auto-schedules experiments for failure modes that keep recurring in production.
Adversarial agent mode	Two competing agents: one breaks things, one defends. The attacker finds what the defender missed — a live war game.
Cost and freeze awareness	Integrates cloud cost APIs and deploy freeze calendars so experiments never run at risky times.
Human feedback loop	Engineers edit runbooks. Diffs fed back to improve future LLM output over time.
Blast radius visualiser	Interactive pre-experiment graph showing predicted cascade paths before any fault fires.

4. Fault Injection Catalogue

The agent supports 10 fault injection categories and 20 distinct fault types covering every layer of a modern cloud-native system — from network and infrastructure through application code, authentication, data storage, the Kubernetes control plane, and full-scale regional failures.

4.1 All 10 Categories at a Glance

Category	Status	What it injects	Primary Tools
Network chaos	Original	Latency, packet loss, DNS failure, AZ partition	Chaos Mesh NetworkChaos, tc/netem
Resource exhaustion	Original	CPU spike, memory pressure, disk fill, fd exhaustion	stress-ng, Chaos Mesh StressChaos
Pod and process kill	Original	Random termination, SIGKILL, restart loops, OOM	Chaos Mesh PodChaos, LitmusChaos
Dependency failures	Original	DB blackhole, cache eviction, API timeout, queue pause	Toxiproxy, NetworkChaos
Clock and time skew	Original	NTP drift, future timestamps, leap second, timezone	libfaketime, syscall intercept
Application-level	New	Exception injection, slow queries, bad responses	Byte Monkey, eBPF probe, Envoy filter
Security and auth	New	Cert expiry, token failures, RBAC and IAM breaks	TLS proxy, cert-manager, custom auth proxy
Data and storage	New	Corruption, replication lag, IOPS throttle, partition	Chaos Mesh IOChaos, Toxiproxy
Kubernetes-specific	New	Node drain, etcd stress, scheduler and controller kill	LitmusChaos, K8s API, custom operator
Chaos at scale	New	Region failover, multi-AZ failure, CDN edge outage	AWS FIS, GCP FIS, Route53, DNS

Original to expanded

The original 5 categories cover the most common infrastructure failure modes and are all buildable in Phase 1. The 5 new categories cover the failure modes that cause the hardest-to-diagnose production incidents and are targeted in Phase 2 and 3.

4.2 Application-Level Faults (New)

- Exception injection — HTTP 500/503/429 on configurable routes; unhandled exception and panic via bytecode or eBPF; gRPC status code injection
- Slow query injection — Latency on specific DB query patterns; N+1 storm simulation; connection pool exhaustion trigger

- Bad response injection — Malformed JSON/XML/Protobuf; truncated response streams; schema drift simulation

4.3 Security and Auth Faults (New)

- Certificate expiry — Expired TLS cert injection; mTLS handshake failure; certificate chain validation error
- Auth token failures — JWT expiry injection; OIDC discovery blackhole; OAuth token introspection 401 responses
- RBAC and permission chaos — Remove K8s ClusterRole bindings; revoke AWS IAM role; Secrets Manager access denial

4.4 Data, Kubernetes, and Scale Faults (New)

- Data corruption — Bit flip injection on file paths; checksum mismatch in object storage; truncate file writes mid-operation
- Replication lag — Artificial delay 1s to 5 minutes; block replica sync then release; split-brain simulation
- Node drain — kubectl drain; node cordon; NotReady injection; Spot/Preemptible termination notice simulation
- etcd stress — Write latency injection; leader election disruption; API server rate limiting; watch event flood
- Region failover — Block egress to target region; DNS failover; cross-region DB promotion timing validation
- Multi-AZ failure — Network partition to AZ subnets; terminate all instances in AZ; AZ storage volume failure
- CDN failure — Origin timeout injection; cache purge thundering herd; edge node routing failure

5. Scenario Definition — Developer and Agent Combined

Scenario definition is the process of deciding what to test and why. The Chaos Engineering Agent supports three input modes — developer-defined, agent-suggested, and autonomous — and they are designed to work together. The best experiments come from combining both human context and machine-generated systematic coverage.

The key insight

The developer brings context the agent cannot see: business logic, upcoming launches, known risky areas. The agent brings systematic coverage the developer would never manually design: topology-aware suggestions, incident replay, and adversarial multi-hop failure chains. Together they produce experiments neither would create alone.

5.1 The Three Input Modes

Mode 1 — Developer defines the scenario

The engineer types a natural language description. The LLM translates it into a fully structured experiment plan with fault type, target scope, duration, intensity ramp, and rollback procedure. No YAML, no domain expertise needed.

- Input via chat interface, Slack bot, or REST API
- Example: 'Test what happens if the payments DB loses connectivity during checkout'
- Agent validates safety constraints before generating the plan
- Developer reviews the plan and approves — or refines it in conversation

Mode 2 — Agent suggests, developer approves

The agent proactively surfaces scenario suggestions based on service topology, recent code changes, past incidents, and historical experiment gaps. The developer picks, tweaks if needed, and approves. Agent runs it.

- Topology-aware: 'Your checkout service has no circuit breaker on the inventory API — suggested: inventory timeout during peak load'
- Change-aware: 'PR #441 changed retry logic — suggested: stress the new retry path under 500ms latency'
- Incident replay: 'Last Tuesday's outage was caused by Redis eviction — suggested: cache miss storm replay'
- Gap-aware: 'This service has never been tested against a DB failover — suggested: connection blackhole for 5 minutes'

Mode 3 — Agent runs autonomously

For CI/CD pipelines and game days, the agent generates and executes scenarios without a developer in the loop — within a pre-approved blast radius policy. Every autonomous experiment still produces findings that a human reviews.

- CI/CD gate: generates 2-3 targeted faults for the service changed in a PR, runs them in staging automatically
- Game day: generates a full multi-service experiment sequence based on current resilience scorecard gaps
- All autonomous experiments bounded by: max blast radius policy, staging-only unless approved, no scale-level faults

5.2 LLM Scenario Generation Capabilities

When the agent generates scenarios, it draws on six input sources and applies reasoning across all of them simultaneously:

Input source	What the agent does with it
Service topology	Reads K8s manifests and Istio mesh to find unprotected dependencies, single points of failure, and cascade paths
Historical experiments	Knows what has already been tested and generates experiments that fill the coverage gaps
Incident replay	Reads PagerDuty or Opsgenie alerts and post-mortem documents to generate the exact experiment that would have caught the issue
Code change analysis	Reads PR diffs to identify what changed and generate experiments that stress the specific changed logic
Compliance requirements	Reads SLA and compliance documents to generate the experiments needed to evidence availability controls
Adversarial reasoning	Reasons about the system like an attacker — thinking 2-3 failure hops ahead to design multi-step cascade scenarios

5.3 How Developer and Agent Work Together

The hybrid model produces better experiments than either input alone. The developer sets the intent and context. The agent provides depth, systematic coverage, and scenario combinations the developer would not manually design.

Developer provides	Agent provides	Resulting experiment
Developer input	Agent input	Combined scenario
Focus on checkout during the sale launch	Topology shows 3 unprotected upstream dependencies	Upstream inventory API timeout + cache miss simultaneously during high traffic — the exact conditions of the last outage
Test the new payment retry logic	PR #441 changed the exponential backoff configuration	Inject 500ms upstream latency at 50% packet loss to stress the specific new retry path under realistic degraded conditions

Developer provides	Agent provides	Resulting experiment
Replay last Tuesday's Redis incident	Historical profile shows cache recovery takes 45s under load	Cache eviction + simultaneous traffic spike + DB connection pool at 80% — full recreation of the incident with compound faults

Scenario library

Every experiment that runs — developer-defined or agent-generated — is stored in a searchable scenario library. Engineers can ask: 'Has anyone tested DB failover during a traffic spike before?' The LLM searches semantically, finds the closest match, and offers to re-run it with updated parameters for the current service.

6. Usage Modes and Remediation

6.1 Three Usage Modes

Mode	Cadence	How it works
Developer-triggered	Daily use	Engineer describes scenario via chat or Slack. Agent plans, runs, and reports. Under 5 minutes from prompt to first fault.
CI/CD gate	Every PR merge	Lightweight smoke test runs automatically in staging. PR comment shows resilience score delta. Pipeline fails if score drops more than 10 points.
Game day	Once per sprint	Coordinated multi-service chaos event. Experiments run in sequence. Agent narrates live. Full debrief report produced automatically.

6.2 The 7-Stage Remediation Loop

Remediation is where the agent delivers its deepest value. It does not just show what broke — it diagnoses root causes, prescribes specific fixes with concrete values, and creates actionable tickets automatically.

Stage	What happens	Timing
1 — Experiment runs	Fault injected at configured scope. Adaptive intensity starts at 10% and ramps based on live metrics.	T+0s
2 — Guard monitors	Evaluates error rate, p99 latency, and saturation against baseline every 15 seconds.	Every 15s
3 — Breach detected	Abort event fires automatically. Breach logged with exact metric values and timestamp.	Within 15s
4 — Auto rollback	Chaos manifest deleted. Agent polls metrics until baseline is restored.	Within 8-12s
5 — LLM diagnosis	Root cause analysis. Compares digital twin prediction vs actual outcome. Identifies weakest link.	1-3 min
6 — Prescription	Specific fixes: named config changes, code patterns, K8s settings with concrete values.	3-5 min
7 — Tickets created	One GitHub Issue or Jira ticket per finding — pre-filled, pre-tagged, ready to assign.	Within 5 min

6.3 Sample Remediation Output

Finding	Prescription	Severity
DB connection pool exhausted in 12s	Increase pool size 10 to 25. Add 3s connection timeout and 1s pool wait timeout.	Critical

Finding	Prescription	Severity
No circuit breaker on payments API	Add Resilience4j: 50% failure threshold, 10s sliding window, 30s open state.	Critical
Pod restart took 45s, no readiness probe	Add readiness probe: httpGet /health, initialDelaySeconds: 5, periodSeconds: 5.	High
Cache miss caused 10x DB load spike	Add fallback cache layer. Set stale-while-revalidate TTL of 30 seconds.	High
No retry on upstream timeout	Add exponential backoff: 3 attempts, 100ms base, 2x multiplier, 2s max.	Medium
Alert fired 3 min after fault injected	Tighten Prometheus rule: evaluation window 5m to 1m for error rate alerts.	Medium

Auto-generated runbooks

Every experiment produces a structured runbook pushed to Confluence or GitHub Wiki automatically. The 9-section structure covers: hypothesis, experiment config, steady-state baseline, execution timeline, observations, findings, prescriptions, resilience score before and after, and the recommended next experiment.

7. Technology Stack and Build Plan

7.1 Technology Stack

Component	Technology
Agent orchestration	Python 3.12, FastAPI, Celery, Redis
LLM reasoning	Anthropic Claude API (claude-sonnet-4), LangChain, CrewAI
Fault injection — Kubernetes	Chaos Mesh 2.x, LitmusChaos 3.x, Toxiproxy
Fault injection — cloud	AWS Fault Injection Simulator, GCP Fault Injection
Digital twin	NetworkX (dependency graph), custom Monte Carlo simulation engine
Observability	Prometheus, Grafana, OpenTelemetry SDK
Data storage	PostgreSQL (experiments and audit trail), Redis (state cache)
Alerting	PagerDuty API, Slack Bolt SDK
CI/CD	GitHub Actions, GitLab CI, ArgoCD
Reporting	Markdown generation, Confluence REST API, WeasyPrint PDF
Infrastructure	Kubernetes 1.28+, Helm 3, Terraform

7.2 Build Roadmap — 3 Phases, 18 Weeks

Phase 1 — Foundation (Weeks 1-5)

Deliverable

Working CLI: single chaos experiment with auto-rollback and Prometheus metric collection. Demo-able end-to-end.

Week	Work
Week 1	kind/minikube cluster. Google Online Boutique as chaos target. Chaos Mesh via Helm. First manual pod-kill experiment.
Week 2	Python orchestration layer. FastAPI REST API. Experiment state machine: pending → running → aborting → complete.
Week 3	Prometheus metric collection. 5-minute baseline capture. Steady-state guard with auto-abort trigger.
Week 4	Idempotent rollback mechanism. Pre-validated rollback before every fault. Full abort → rollback → recovery cycle tested.
Week 5	CLI end-to-end. Pod kill, network latency, dependency blackhole working. Metrics logged. First demo-able V0.

Phase 2 — Intelligence Layer (Weeks 6-12)

Deliverable

Full web UI and chat interface. Digital twin simulation, adaptive intensity, LLM scenario generation, and auto runbooks working end-to-end.

Week	Work
Week 6-7	Claude API integration. LangChain reasoning chain: NL scenario → experiment plan → safety validation → JSON output.
Week 8	Adaptive intensity control loop. Ramp logic with 30-second metric evaluation. Configurable speed tiers.
Week 9	Dependency graph from K8s manifests and Istio topology. Basic blast radius prediction per service.
Week 10	Digital twin simulation. Monte Carlo scoring. Pre-mortem narrative generation. Split-screen demo: twin vs real cluster.
Week 11	Agent-generated scenario suggestions. Incident replay from PagerDuty. Scenario library with semantic search.
Week 12	Web UI. Slack bot and /chaos commands. PagerDuty integration. Auto-runbook generation and Confluence push.

Phase 3 — Production Hardening (Weeks 13-18)

Deliverable

Production-trustworthy system with human approval gates, CI/CD chaos gate, multi-cloud support, and Chaos DNA profiling. Helm chart for one-command deployment.

Week	Work
Week 13	Human-in-the-loop approval gate for production. Slack one-click approve/reject. Logged immutably.
Week 14	GitHub Actions chaos gate. PR resilience score comment. Pipeline status check on score delta.
Week 15	AWS FIS integration. Multi-cluster support. Unified experiment API across K8s and cloud targets.
Week 16	Chaos DNA profiling MVP. Profile builder from experiment history. New-service auto-recommendations.
Week 17	Human feedback loop on runbooks. Diff capture. Prompt improvement from collected feedback.
Week 18	Helm chart. Load testing orchestrator. Full documentation site. v1.0 release.

7.3 Success Metrics

Metric	Target
Safety	Zero unintended production outages caused by the agent in 6 months
Adoption	10+ engineering teams running at least 1 experiment per sprint
LLM accuracy	Experiment plan accepted without modification more than 80% of the time
Rollback reliability	Successful automatic rollback in more than 99.5% of abort events
Cycle time	Scenario description to first fault injected in under 5 minutes
Resilience trend	Average service resilience score improves by more than 15 points over first 6 months

8. Executive Summary

This section provides a complete one-page summary of the Chaos Engineering Agent project — what it is, what it does, and why it matters.

What it is

The Chaos Engineering Agent is an AI-powered autonomous system that deliberately breaks distributed systems in controlled, safe ways to discover resilience weaknesses before they become real production outages. It is the first chaos engineering tool that combines a digital twin simulation layer, LLM-driven scenario generation, adaptive fault intensity, and a full automated remediation pipeline — making chaos engineering accessible to every engineering team, not just those with dedicated SREs.

What it does

Capability	Detail
Breaks things safely	Injects 20 types of faults across 10 categories — network, resources, pods, dependencies, application code, security, data, Kubernetes control plane, and full regional failures — all with automatic rollback and blast radius controls.
Thinks before it breaks	Builds a digital twin of the system, runs 1,000+ Monte Carlo simulations to predict blast radius, and generates a written pre-mortem before any real fault fires.
Defines its own scenarios	The LLM generates experiment scenarios from service topology, incident history, PR diffs, compliance requirements, and adversarial reasoning — not just from what engineers remember to ask for.
Combines developer and agent input	Developer provides intent and business context. Agent provides systematic coverage and multi-hop failure chains. Together they produce experiments neither would create alone.
Fixes what it finds	Diagnoses root causes, prescribes specific fixes with named parameters and concrete values, and creates pre-filled GitHub Issues or Jira tickets — automatically, within 5 minutes of an experiment completing.
Gets smarter over time	Every service builds a Chaos DNA profile. The digital twin improves as prediction gaps are measured. The scenario library grows. The resilience scorecard tracks improvement sprint over sprint.

Why it matters

Teams that practise chaos engineering proactively reduce mean time to recovery by 30-50% and prevent an estimated 70% of repeat incidents. Today, that practice requires deep domain expertise and significant manual effort — putting it out of reach for most engineering teams. The Chaos Engineering Agent removes both barriers: the LLM layer handles expertise, and the automation layer handles effort. The result is a system that turns chaos engineering from a

quarterly manual exercise into a continuous, AI-driven practice that runs on every PR, every sprint, and every game day — and that gets measurably better with every experiment it runs.

Is it buildable?

Yes. Every component uses existing, production-grade open-source tooling. A solo developer can ship a working V1 — fault injection, LLM planning, safety guard, and basic reporting — in 6-8 weeks. A team of 2-3 engineers can ship the full 18-feature vision including digital twin, hybrid scenario definition, and CI/CD integration in 4-5 months. The full build is structured across 3 phases and 18 weeks with clear weekly deliverables and a demo-able milestone at the end of each phase.

The one-line pitch

It turns chaos engineering from a once-a-quarter manual exercise done by expert SREs into a continuous, automated, AI-driven practice that any engineering team can run — and that gets smarter with every experiment it runs.